

unuser

UniqueUser — Manual do Usuário

Sincronização manual entre máquinas Debian, com cofre cifrado (zero-knowledge) · v1.0.1

1. O que é o unuser

O **unuser** sincroniza seus documentos entre várias máquinas Debian através de um servidor central que funciona como um **cofre cego**: ele guarda apenas dados **já cifrados** e nunca tem acesso ao conteúdo, aos nomes dos arquivos nem à estrutura de pastas. A sincronização é **manual** — você decide o que enviar, receber ou apagar, por status e ações na interface.

São dois programas:

- **unuserd** — o servidor (cofre cego). Fica sempre ligado num nó central; escuta só em `localhost` (o acesso de fora é via Tor).
- **unuser** — o cliente, que você usa em cada máquina. Tem uma **interface gráfica** (estilo Explorer, tema escuro) e uma **CLI** equivalente.

2. Como funciona (resumo)

A confidencialidade depende de **dois fatores**, que **nunca** saem da sua máquina nem chegam ao servidor:

- **Passphrase** — uma senha que você digita (ou passa por variável de ambiente).
- **Keyfile** — um arquivo de alta entropia (segredo) guardado localmente.

Dos dois fatores deriva-se a chave-mestra (Argon2id + HKDF) e dela as chaves de cada finalidade. Cada arquivo é cifrado com uma chave própria (AES-256-GCM); o servidor só vê blocos cifrados, identificadores opacos e metadados de versão. Detalhes técnicos na *Especificação* (`doc/especificacao-unuser.html`).

Perdeu a passphrase OU o keyfile? O cofre fica irrecuperável — não há recuperação possível (é o objetivo do zero-knowledge). Veja a seção *Segurança e backup*.

3. Requisitos

- **Debian 13 (trixie)** (recomendado), **Python ≥ 3.11**.
- Servidor: `python3` + `python3-cryptography`.
- Cliente (CLI): `python3-cryptography`, `python3-argon2` (o `blake3` vem embarcado no `.deb`).
- Cliente (GUI): `python3-pyside6.qtwidgets` e, em Qt 6.5+, `libxcb-cursor0`.
- Acesso remoto entre máquinas: **Tor** (sem VPN).

4. Instalação

4.1 Servidor — pacote `.deb`

```
sh packaging/build-deb.sh                # gera dist/unuserd_1.0.1_all.deb (sem
root)
sudo apt install ./dist/unuserd_1.0.1_all.deb # cria o usuário 'unuser' e
/var/lib/unuser/vault
sudoedit /etc/default/unuserd             # storage / host / porta (e mTLS, se
usar)
sudo systemctl start unuserd
```

A instalação **habilita** (não inicia) o serviço, que roda como o usuário `unuser` com endurecimento `systemd`. O `unuserd` escuta só em `localhost` — quem publica o serviço para fora é o Tor (seção 9).

4.2 Cliente — pacote `.deb`

```
sh packaging/build-deb-client.sh          # gera dist/unuser_1.0.1_amd64.deb (sem
root)
sudo apt install ./dist/unuser_1.0.1_amd64.deb # puxa python3-cryptography/-argon2
```

Para a interface gráfica:

```
sudo apt install python3-pyside6.qtwidgets libxcb-cursor0
```

O `blake3` não existe no `apt`, então vem **embarcado** no pacote do cliente (por isso ele é `amd64` e fixado ao Python 3.13 do Debian 13). O CLI de sincronização funciona sem o `PySide6`.

4.3 A partir do código (desenvolvimento)

```
python3 -m venv .venv
.venv/bin/python -m pip install -e ".[dev]"      # runtime + pytest
.venv/bin/python -m pip install -e ".[gui]"      # opcional: PySide6 (GUI)
```

Em máquinas **sem ensurepip/python3-venv** (e sem sudo):

```
python3 -m venv --without-pip .venv
curl -sS -o /tmp/get-pip.py https://bootstrap.pypa.io/get-pip.py
.venv/bin/python /tmp/get-pip.py
.venv/bin/python -m pip install -e ".[dev]"
```

4.4 Interface gráfica (PySide6) — atalho

O script `src/client/run-gui.sh` instala o extra gui e, em Debian 13/Qt 6.5+, baixa `libxcb-cursor` **sem root** num cache local (`~/.cache/unuser/lib`) antes de abrir a janela:

```
UNUSER_PASSPHRASE='sua-senha' src/client/run-gui.sh
```

5. Configuração

Nenhum arquivo de configuração contém segredos (passphrase/keyfile ficam de fora).

5.1 Keyfile e passphrase (os dois fatores)

Crie um keyfile de alta entropia e proteja-o:

```
mkdir -p ~/.config/unuser
head -c 64 /dev/urandom > ~/.config/unuser/keyfile
chmod 600 ~/.config/unuser/keyfile
```

- Keyfile: `--keyfile <caminho>` ou variável `UNUSER_KEYFILE`.
- Passphrase: digitada no prompt, ou variável `UNUSER_PASSPHRASE` (útil para a GUI).

O mesmo keyfile precisa estar em **todas** as máquinas do cofre — copie-o por um canal seguro; **não** gere um novo em cada máquina (senão o cofre não abre lá).

5.2 Conexão — ~/.env

```
UNUSER_VAULT_ID=v:default
UNUSER_CONN_MODE=direct          # direct (LAN/localhost) ou tor
UNUSER_DIRECT_HOST=127.1.0.1
UNUSER_DIRECT_PORT=8080
# modo tor:
UNUSER_TOR_ONION=
UNUSER_TOR_PORT=8080
UNUSER_TOR_SOCKS=127.1.0.1:9050
```

5.3 O que entra no cofre — ~/.config/unuser/dirs.json

```
{
  "default_dirs": [
    { "path": "/home/voce/Documents", "recursive": true }
  ],
  "extra_items": []
}
```

Cada arquivo recebe um *nome no cofre* = nome da pasta-raiz + caminho relativo (ex.: `/home/voce/Documents/x.txt` → `Documents/x.txt`), estável entre máquinas com `$HOME` diferente. Você também pode adicionar/gerenciar pastas pela GUI (botão **Pastas...**), ou deixar que o botão **Adicionar arquivo** registre a pasta automaticamente (seção 7).

5.4 Exclusões — ~/.unuserignore

Globs estilo `.gitignore`. Já existem padrões de segurança embutidos (mesmo sem o arquivo): `.env`, `*.key`, `*.pem`, `.ssh/`, `.gnupg/`, `*.tmp`, `*.swp`, `.git/`.

6. Primeiro uso (CLI)

1. Suba o servidor (num terminal ou via systemd / `src/server/run.sh`):

```
src/server/run.sh          # cofre em ~/.local/share/unuser-vault,
127.1.0.1:8080
```

2. No cliente, exporte os fatores e use os subcomandos:

```
export UNUSER_KEYFILE=~/.config/unuser/keyfile
```

```
export UNUSER_PASSPHRASE='sua-senha'

unuser status                # compara local x servidor
unuser send Documents/x.txt  # envia tudo que mudou (ou: send
unuser receive Documents/x.txt # baixa do cofre
unuser delete Documents/x.txt # apaga (tombstone no servidor + remove local)
```

Na primeira vez, o cofre gera o salt do Argon2 e deriva as chaves; depois valida passphrase/keyfile abrindo o manifesto.

7. A interface gráfica

Abra com `unuser gui` (ou `src/client/run-gui.sh`). É um Explorer de duas áreas, em tema escuro:

- **Esquerda — Pastas:** a árvore de pastas do cofre. A expansão de cada nó e a pasta selecionada são **lembradas entre sessões**.
- **Direita — Arquivos:** a lista dos arquivos **diretamente** na pasta selecionada (não recursivo), cada um com sua cor de status.

Cores de status:

- Em sincronia Modificado no local Modificado no servidor
- Conflito Só local Só no servidor

Botões (barra e menu de contexto):

Botão	O que faz
Atualizar	Recompara local x servidor.
Enviar →	Envia ao cofre os arquivos selecionados (cria nova versão).
← Receber	Baixa do cofre os arquivos selecionados.
Apagar	Apaga (tombstone no servidor + remove a cópia local).
Adicionar arquivo	Escolhe um arquivo e o envia. Se ele estiver fora das pastas sincronizadas mas dentro de <code>~/</code> , a pasta dele é registrada automaticamente (ou, se o arquivo estiver

direto em `~/`, ele entra como item avulso — a home inteira nunca é sincronizada).

Pastas... Tela para listar, adicionar e remover pastas sincronizadas e itens avulsos.

As operações de rede rodam fora da thread da interface — a janela não trava em conexões lentas (ex.: via Tor). O estado da árvore é gravado em `~/ .local/data/unuser/`.

8. Scripts de conveniência

Script	Uso
<code>src/server/run.sh</code>	Sobe o <code>unuserd</code> (variáveis <code>UNUSERD_STORAGE/HOST/PORT</code> ; mTLS via <code>UNUSERD_TLS_CERT/KEY/ALLOW</code>).
<code>src/client/run.sh</code> <code><cmd></code>	Cliente CLI; gera o keyfile na 1ª vez e repassa <code>status send receive delete gui</code> .
<code>src/client/run-gui.sh</code>	Instala PySide6 + <code>libxcb-cursor0</code> (sem root) e abre a GUI.

9. Acesso remoto por Tor

Entre máquinas, o transporte é por **Tor** (sem VPN): o `unuserd` só escuta TLS em `localhost`; o daemon `tor` publica um *Onion Service*; o cliente conecta ao endereço `.onion` pelo SOCKS5, com o mTLS por dentro do túnel. Configure no cliente `UNUSER_CONN_MODE=tor`, `UNUSER_TOR_ONION=...` e `UNUSER_TOR_SOCKS=127.1.0.1:9050`. Passo a passo em `doc/operacao-tor.md`.

10. Segurança e backup

- Faça **backup do keyfile** (offline, em local seguro) e memorize a passphrase. Sem qualquer um dos dois, o cofre é irrecuperável.
- O keyfile deve ser idêntico em todas as máquinas; copie-o por canal seguro.
- O servidor é cego: vê apenas dados cifrados, ids opacos, o salt do Argon2 (público) e os metadados de versão.
- Mantenha o keyfile com permissão `600` e fora de pastas sincronizadas (os padrões de exclusão já evitam `*.key/`, `.ssh` etc.).

11. Solução de problemas

Sintoma	Causa / solução
GUI não abre: <i>"Could not load the Qt platform plugin xcb"</i>	Falta <code>libxcb-cursor0</code> : <code>sudo apt install libxcb-cursor0</code> (ou use <code>run-gui.sh</code> , que resolve sem root).
<i>passphrase/keyfile incorretos (ou salt divergente)</i>	Confira a passphrase e se o keyfile é o mesmo das outras máquinas.
"fora das pastas sincronizadas"	O arquivo está fora de <code>~/</code> . Mova-o para dentro de <code>~/</code> ou adicione a pasta em Pastas... / <code>dirs.json</code> .
GUI sem display	A interface precisa de um servidor gráfico (X11/Wayland). Use a CLI em servidores headless.

12. Onde fica o quê

Caminho	Conteúdo
<code>~/env</code>	Conexão (modo, host/porta, Tor).
<code>~/config/unuser/dirs.json</code>	Pastas e itens sincronizados.
<code>~/config/unuser/index.db</code>	Índice local (descartável).
<code>~/config/unuser/keyfile</code>	Keyfile (segredo) — sugestão de local.
<code>~/unuserignore</code>	Exclusões (globs).
<code>~/local/data/unuser/gui-state.json</code>	Estado da GUI (expansão/seleção).
<code>/var/lib/unuser/vault</code>	Cofre no servidor (blobs + manifesto cifrados).